

How to configuration your google voice

In order to improve use voice interaction experience. We introduced Google dialogflow in our system. We support and encourage our clients to develop their own voice library. This document is to tell you how to rich your robot's voice content.

Contents

1.	To register a google account.....	3
2.	Login dialogflow.....	3
3.	How the dialogflow works.....	5
4.	Key words of Agent.....	6
5.	Create your first Agent.....	7
7.	Create your first intent.....	8
8.	Extract data with entities.....	9
9.	Create your own entities.....	11
10.	Manage state with contexts.....	13
10.1	Add contexts to conversational state.....	13
10.2	Intent matching with follow-up intents.....	14
10.3	Try it out.....	14
10.4	Intents and contexts.....	15
10.5	Contexts and parameters.....	16

1. To register a google account

Skip this step, if you already have a google account.

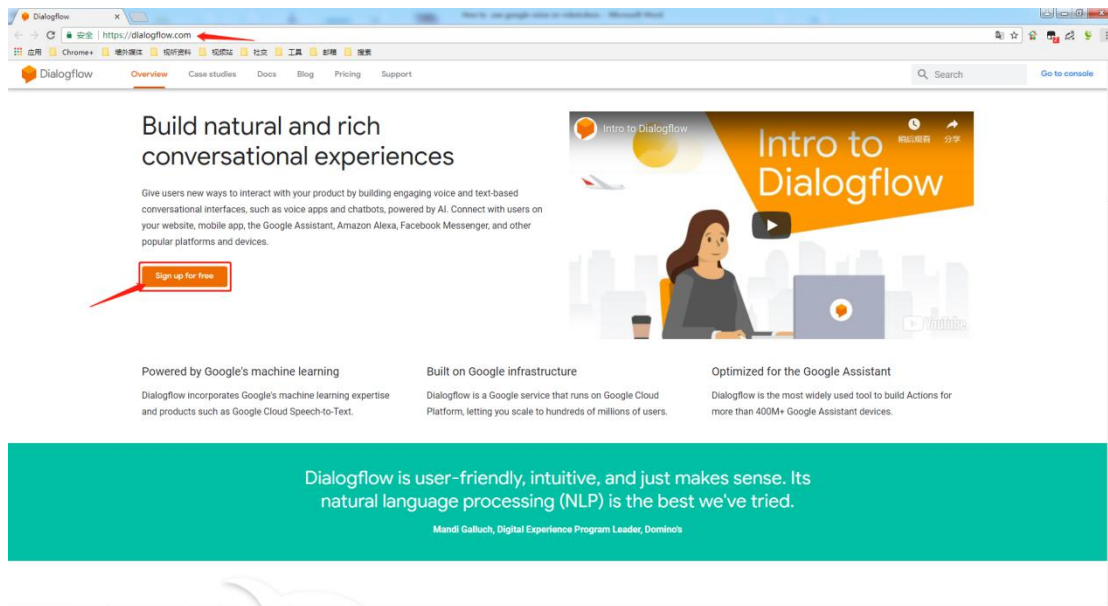
As this service is provided by google so a google account is needed. You can register a google account from google website.

<https://accounts.google.com/SignUp?continue=https%3A%2F%2Fwww.google.com%2F&hl=en&gmb=exp&biz=false>

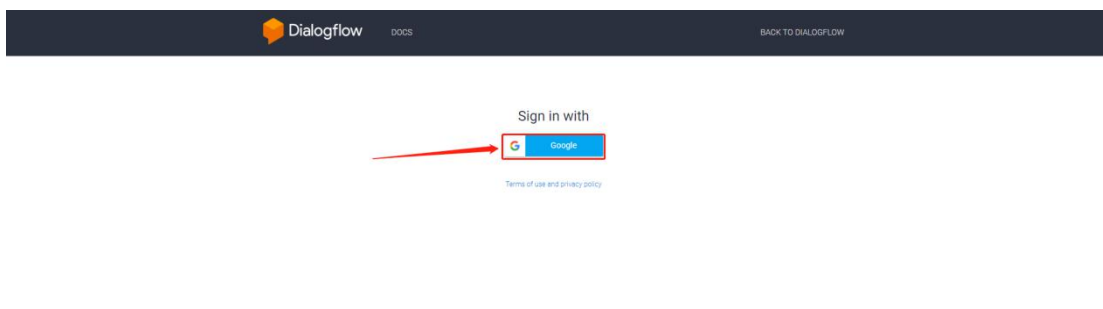
2. Login dialogflow

Now you have a google account, you can login dialogflow with your google account:

www.dialogflow.com



Click "Sign up for free" to login with your google account.



Sign in with Google

Choose an account

to continue to [Dialogflow](#)



Signed out.

2

Use another account

To continue, Google will share your name, email address and profile picture with Dialogflow. Before using this app, you can review Dialogflow's [privacy policy](#) and [terms of service](#).

English (United Kingdom)

Help

Privacy

Terms

Please review your account settings

Country or territory *

United Kingdom

Email preferences

Stay up-to-date with occasional emails from our team

☒ News and tips

Learn about new features, enhancements and tips

☒ Feedback and testing

Participate in surveys and pilots to improve Dialogflow

Terms of Service *

☒ Yes, I have read and accept the agreement.

By proceeding and clicking the button below, you agree to adhere to the [Terms of Service](#).

Additionally, you may have access to certain Firebase services. You agree that your use of Firebase services will adhere to the applicable [Firebase Terms of Service](#). If you integrate any apps with Firebase on this project, by default, your Firebase Analytics data will enhance other Firebase features and Google products. You can control how your Firebase Analytics data is shared in your Firebase settings at anytime.

ACCEPT

Sign in with Google

Dialogflow wants to access your Google Account

 dylan.r.dang@gmail.com

This will allow Dialogflow to:

View and manage your data across Google Cloud Platform services

1

View and manage your Actions on Google.

1

View and manage your Google Assistant voice commands, dialog and grammar

1

Submit log data for your projects

1

Make sure you trust Dialogflow

You may be sharing sensitive info with this site or app. Learn about how Dialogflow will handle your data by reviewing its [terms of service](#) and [privacy policies](#). You can always see or remove access in your [Google Account](#).

[Learn about the risks](#)

Cancel

Allow

English (United States)

Help

Privacy

Terms

Dialogflow

Create Agent

Docs

Support

Account

Logout

Welcome to Dialogflow!

Don't know where to begin? Let us help you get started.

Get started

Now it's time to create your first agent.

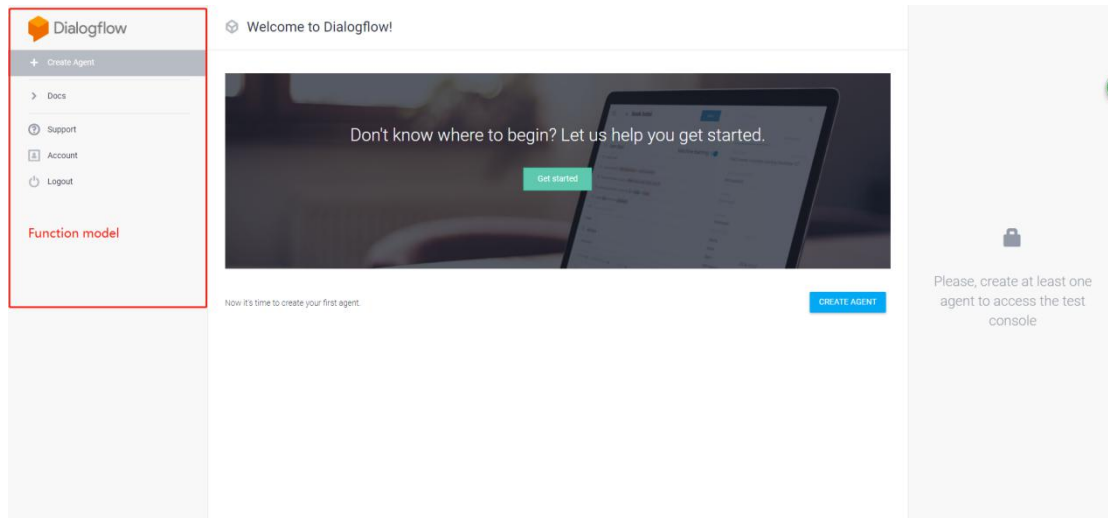
CREATE AGENT

Please, create at least one agent to access the test console

Congratulations ! You login successfully ! You can learn more skills by click “get started”

button.

More details: <https://dialogflow.com/docs/getting-started/create-account>



3. How the dialogflow works

Let me introduce how dialog flow works briefly, before we start to create dialogflow project.

Dialogflow lets you build conversational interfaces on top of your products and services by providing a powerful natural language understanding (NLU) engine to process and understand natural language input. This document goes over how Dialogflow works and how it can help you create conversational user interfaces that delight users.

Traditional computer interfaces require structured and predictable input to function properly, which makes the use of these interfaces unnatural and sometimes difficult. If users can't easily figure out this structured input, they'll have a hard time figuring out what to do.

For example, consider an easy user request like "What's the forecast like today?". Other users might also ask:

"What's the weather like right now?"

"What's the temperature like in San Francisco tomorrow?"

"What will the weather be like on the 21st?"

Even with this simple question, you can see that conversational experiences are hard to implement. Interpreting and processing natural language requires a very robust language parser that's capable of understanding the nuances of language.

Your code would have to handle all these different types of requests (and potentially many more) to carry out the same logic: looking up some forecast information for a time and location. For this reason, a traditional computer interface would tend to force users to input a well-known, standard request at the detriment of the user experience, because it's just easier.

However, Dialogflow lets you easily achieve a conversational user experience by handling the natural language understanding (NLU) for you. When you use Dialogflow, you create agents that can understand the vast and varied nuances of human language and translate that to standard and structured meaning that your apps and services can understand. Let's take a look at how Dialogflow might handle the previous examples for weather forecast requests.

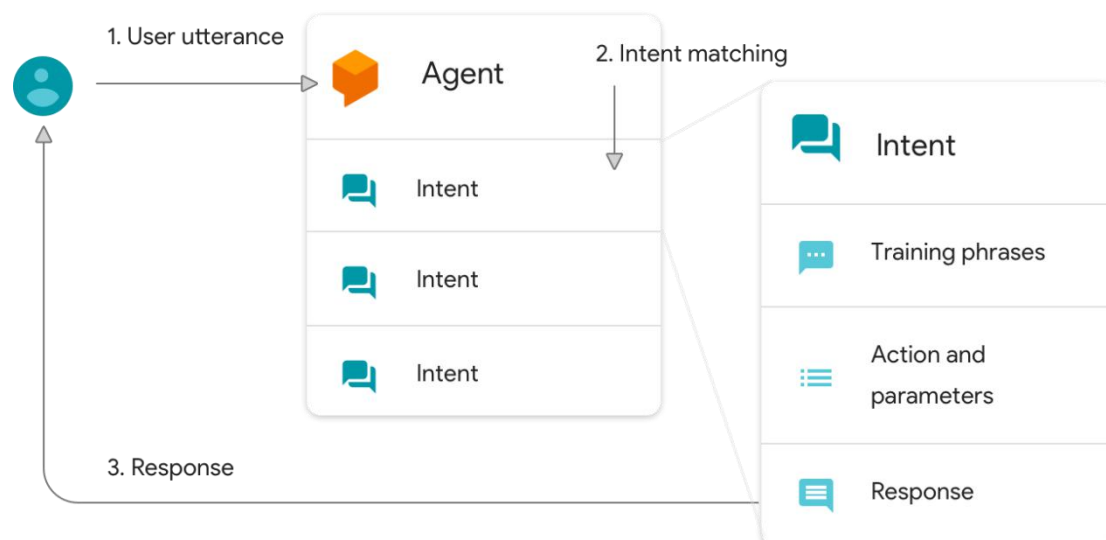


Figure 1. Example of how Dialogflow handles a user utterance.

To look up a weather forecast, you might need a few pieces of information, like the time users want the forecast for and their location. However, as we previously mentioned, different users might request a forecast in different ways. Dialogflow can understand these differences and translate them to a standard user intent to get the forecast. It can then parse the user's request for the pertinent data you need to fulfill the request. In this case, that's the user's desired time and location for the weather forecast. Finally, you can use this data to look up the weather with a public REST API and return the weather to the user in the form of a response.

4. Key words of Agent

Now you should have a skeleton in your mind about the dialog flow working model. You must be want to start your project soon. But there still have some noun need to be explained. I'll wrap up the tiresome verbosity as soon as possible.

Key words of Agent:

- **Intent name:** The name of the intent. The intent name is passed to your fulfillment and identifies the matched intent.
- **Training phrases:** Defines example phrases of what users can say. Dialogflow uses these training phrases and naturally expands them to many more similar phrases to build a language model that accurately matches user input. Through further training and machine learning, Dialogflow builds a more robust and varied language model to better match user input.
- **Action and Parameters:** To improve an intent's language model, you can also annotate your training phrases with *entities*, or categories of data that you want Dialogflow to match. This lets you tell Dialogflow that you want a particular type of input and to not just match the literal input of the user. Dialogflow extracts matched entities as parameters from the training phrases. You can then process these parameters in logic called *fulfillment* to further customize a response to the user. You'll learn about fulfillment later in this document.
- **Responses:** Defines a text, speech, or visual response to the user, which usually prompts users in a way that lets them know what to say next or that the conversation is

ending. To send responses, you can use Dialogflow's built-in response handler or call fulfillment to process the extracted data and return a response back to Dialogflow.

- **Entity type:** Defines the type of information you want to extract from user input. For example, *vegetable* could be the name of an entity type. Clicking **Create Entity** from the Dialogflow console creates an *entity type*. When using the API, the term *entity type* refers to an `EntityType` object.

- **Entity entry:** For each entity type, there are many *entity entries*. Each entity entry provides a set of words or phrases that are considered equivalent.

5. Create your first Agent.

With the above concepts in mind, we can start to build our own agent.

To create a Dialogflow agent:

1. Open a browser and log in to Dialogflow.
2. Click **Create agent** in the left menu.
3. Enter your agent's name, default language, and default time zone, then click the **Create** button.

The screenshot shows the 'Create agent' form in the Dialogflow console. It includes the following fields and annotations:

- Agent name:** A text input field with a red arrow pointing to it labeled '1'.
- CREATE:** A blue button with a red arrow pointing to it labeled '4'.
- DEFAULT LANGUAGE:** A dropdown menu showing 'English - en' with a red arrow pointing to it labeled '2'.
- DEFAULT TIME ZONE:** A dropdown menu showing '(GMT+8:00) Asia/Hong_Kong' with a red arrow pointing to it labeled '3'.
- GOOGLE PROJECT:** A section with the text 'New GCP project will be automatically linked to the agent after saving'.

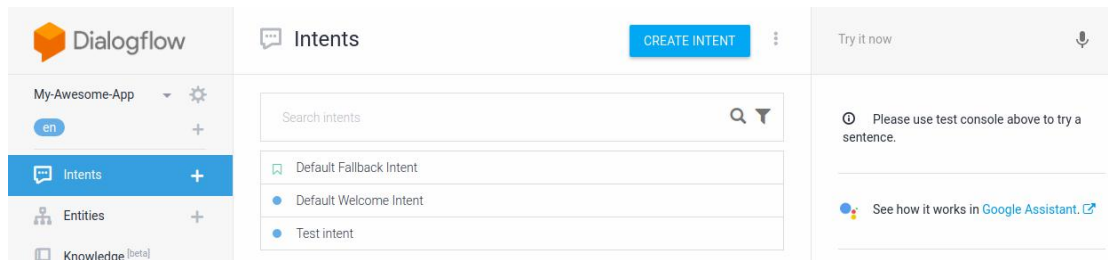
Note: **Agent name can not contain whitespaces**

6. The Dialogflow console

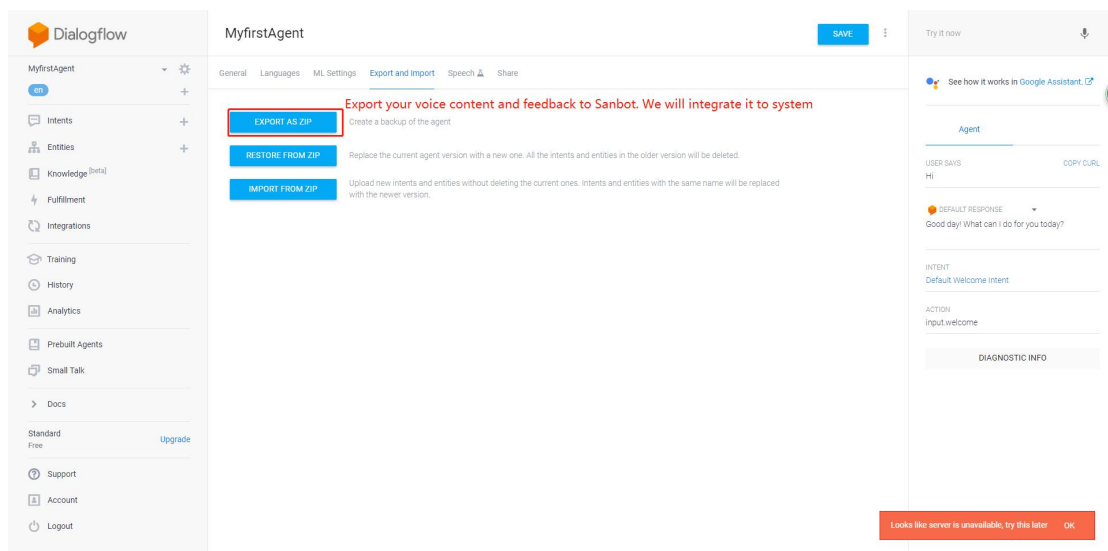
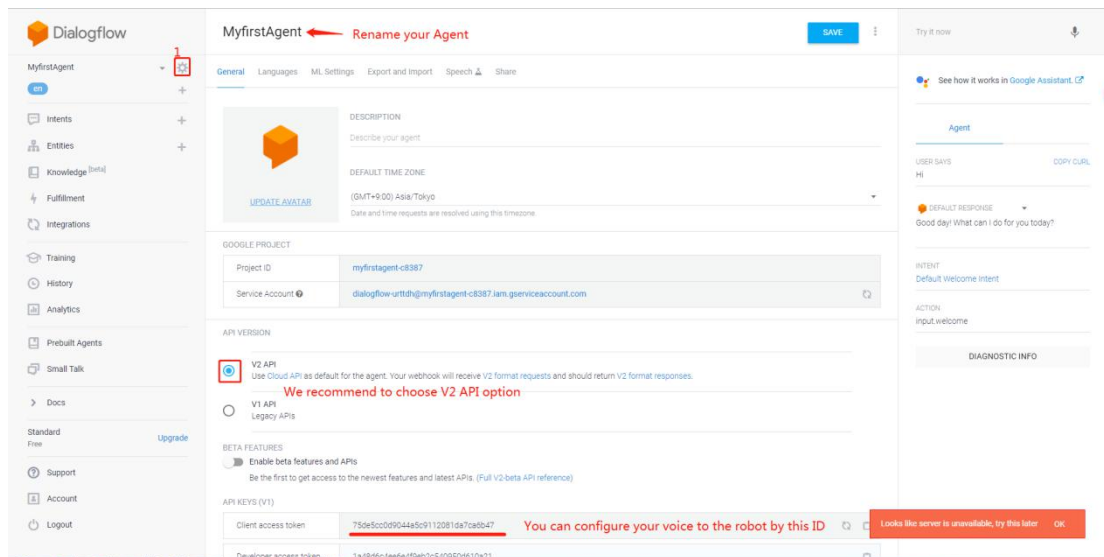
You should now see the Dialogflow console and the menu panel on the left. If you're working on a smaller screen and the menu is hidden, click on the menu  button in the upper left corner. The settings  button takes you to the current agent's settings.

The middle of the page will show the list of intents for the agent. By default, Dialogflow agents start with two intents. Your agent matches the Default Fallback Intent when it doesn't understand what your users say. The Default Welcome Intent greets your users. These can be altered to customize the experience.

The Dialogflow simulator is on the right side of the page. The simulator lets you try out your agent by speaking or typing messages.



Settings:



7. Create your first intent

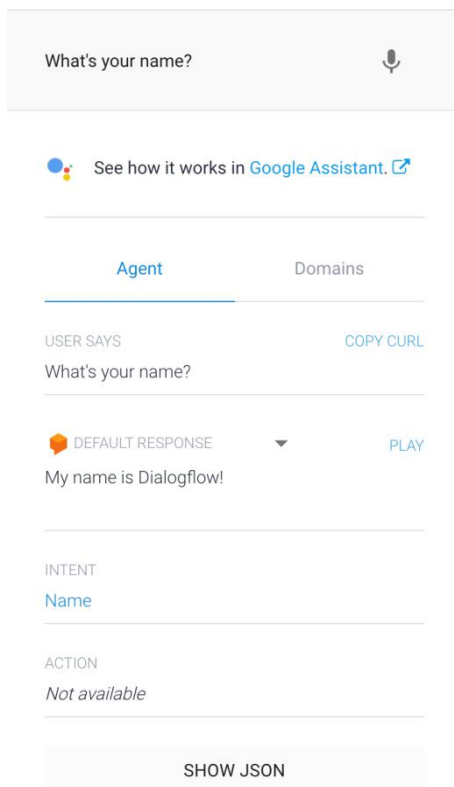
Dialogflow uses intents to categorize a user's intentions. Intents have Training Phrases, which are examples of what a user might say to your agent. For example, someone wanting to know the name of your agent might ask, "What is your name?", "Do you have a name?", or just

say "name". All of these queries are unique but have the same intention: to get the name of your agent.

To cover this query, create an intent by following these steps:

1. Click on the plus add next to **Intents** in the left menu.
2. Add the name "name" into the **Intent name** text field.
3. In the **Training Phrases** section, click on the text field and enter the following, pressing enter after each entry:
 - What is your name?
 - Do you have a name?
 - name
4. In the **Responses** section, click on the text field and enter the following response:
 - My name is Dialogflow!
5. Click the **Save** button.

Try it out!



The screenshot shows the Dialogflow simulator interface. At the top, there is a text input field with the text "What's your name?" and a microphone icon. Below this, there is a link that says "See how it works in Google Assistant." with a small icon. The main interface is divided into two tabs: "Agent" (selected) and "Domains". Under the "Agent" tab, there are three sections: "USER SAYS" with the text "What's your name?" and a "COPY CURL" link; "DEFAULT RESPONSE" with a dropdown arrow and a "PLAY" link, showing the response "My name is Dialogflow!"; and "INTENT" with the text "Name". At the bottom, there is an "ACTION" section with the text "Not available" and a "SHOW JSON" button.

Now try asking your agent for its name. In the simulator on the right, type "What's your name?" and press enter.

Your agent responds to the query correctly, even though your query was a little different from the training phrase ("What's your name?" versus "What is your name?"). It was able to resolve this by using machine learning.

Dialogflow uses training phrases as examples for a machine learning model to match users' queries to the correct intent. The machine learning model checks the query against every intent in the agent, gives every intent a score, and the highest-scoring intent is matched. If the highest scoring intent has a very low score, the fallback intent is matched.

8. Extract data with entities

Add parameters to your intents

Parameters are important and relevant words or phrases in a user's query that are extracted so your agent can provide a proper response. You'll create a new intent with parameters for spoken and programming languages to explore how these can match specific intents and be included in your responses.

1. Create a new intent by clicking on the plus add next to **Intents** in the left menu.
2. Name the intent "Languages" at the top of the intent page.

3. Add the following as Training phrases:

- I know English
- I speak French
- I know how to write in German

4. Click **SAVE**.

• Languages

SAVE

Training phrases ?

Search in user says



” Add user expression

” I know English

” I speak French

” I know how to write in German

Action & parameters ?



Enter action name

REQUIRED ?	PARAMETER NAME ?	ENTITY ?	VALUE	IS LIST ?
☐	language	@sys.language	\$language	☐

Dialogflow automatically detects parameters in your Training phrases that are included in system entities. System entities are entity sets provided by Dialogflow to cover most expected values for things like location, color, and dates.

Below the Training phrases section, Dialogflow fills out some of the table with the information it gathered:

- The parameter is optional (not required).
- The name language (automatically resolved for system entities).
- The entity this entry corresponds to (in this case its the system entity @sys.language).
- The value of \$language (automatically resolved).
- The parameter is not a list.

Note: If entities aren't automatically detected, you can highlight the text in the Training phrase and [manually annotate the entity](#).

Use parameter data in a response

The value of a parameter can be used in your responses. In this case, you can use

\$language in your responses and it will be replaced with the language specified in the query to your agent.

In the Responses section, add the following response and click the SAVE button:

- `Wow! I didn't know you knew $language`

The screenshot shows the Dialogflow console interface. At the top, there's a text input field containing "I know Russian" and a microphone icon. Below this, there's a link that says "See how it works in Google Assistant." with a small icon. The main area is divided into two tabs: "Agent" (selected) and "Domains". Under the "Agent" tab, there's a section for "USER SAYS" with the text "I know Russian" and a "COPY CURL" button. Below that, there's a "DEFAULT RESPONSE" section with a dropdown arrow and a "PLAY" button. The response text is "Wow! I didn't know you knew Russian". At the bottom, there's an "INTENT" section showing "Languages". Below that, there's an "ACTION" section showing "Not available". At the very bottom, there's a table with two columns: "PARAMETER" and "VALUE". The table has one row with "language" in the parameter column and "Russian" in the value column.

PARAMETER	VALUE
language	Russian

In the simulator on the right, query your agent with "I know Russian".

You can see in the bottom of the simulator output that Dialogflow correctly extracted the `language` parameter with the value "Russian" from the query. In the response, you can see "Russian" was correctly inserted where the parameter value was used. Parameters can also be handled in your fulfillment. This is covered later in [Integrating other services with fulfillment](#).

9. Create your own entities

In most cases you'll have specific information you need to gather from users, that isn't covered by Dialogflow's system entities. Developer entities are sets of values and synonyms you provide.

To create an entity, follow these steps:

1. Click on the plus add next to **Entities** in the left menu.
2. Enter "ProgrammingLanguage" for the name of the entity.
3. Click in the left text field and add the following entries:
 - JavaScript
 - Java
 - Python
4. When you enter an entry, pressing tab moves your cursor into the synonym field. Add the following synonyms for each entry:

Entry	Synonym
Javascript	js, ECMAScript
Python	py

5. Click the **Save** button.

Each entity has to have the following:

- a name to define the category (ProgrammingLanguage)
- one or more entries (JavaScript, Python)
- one or more synonyms (js, ECMAScript, py)

Dialogflow can handle simple things like plurality and capitalization, but make sure to add all possible synonyms for your entries. The more you add, the better your agent can determine your entities.

Add your new entities

Now that we've defined our entity for programming languages, add Training Phrases to the "Languages" intent:

1. Click on **Intents** in the left menu, and then click on the "Languages" intent.
2. Add the following as Training phrases:

- I know javascript
- I know how to code in Java

The programming languages in the training phrases should be automatically annotated. This adds the ProgrammingLanguage parameter to the table.

I know how to code in py

See how it works in [Google Assistant](#).

Agent

Domains

USER SAYS

COPY CURL

I know how to code in py

DEFAULT RESPONSE

PLAY

Python is cool.

INTENT

Languages

ACTION

Not available

PARAMETER

VALUE

language

ProgrammingLanguag... Python

REQUIRED ?	PARAMETER NAME ?	ENTITY ?	VALUE
☐	language	@sys.language	\$language
☐	ProgrammingLanguag	@ProgrammingLangu age	\$Programming age

3. In the **Responses** section,
add \$ProgrammingLanguage is cool and then
click the **SAVE** button.

Try it out!

Now, query your agent with the new entity. In the simulator, type I know how to code in py and

press enter.

You can see that Dialogflow correctly extracted the entity `py`, identified it as the Python entity, and inserted the value in the response.

10. Manage state with contexts

A Dialogflow account and agent as well as familiarity with parameters and entities are needed for the tasks on this page. Make sure you create a Dialogflow account, create your first agent and are familiar with extracting data with entities before continuing.

10.1 Add contexts to conversational state

1. Click the plus add next to Intents in the left menu to create a new intent.
2. Name the intent "Languages".
3. Enter the following Training Phrases:
 - I know English
 - I speak French
 - I know how to write in German
4. In the **Response** section, enter the following text response:
 - Wow! I didn't know you knew \$language. How long have you known \$language?
5. Click the Save button.
6. Click on Intents in the left menu.
7. Hover over the "Languages" intent and click on Add follow-up intent:

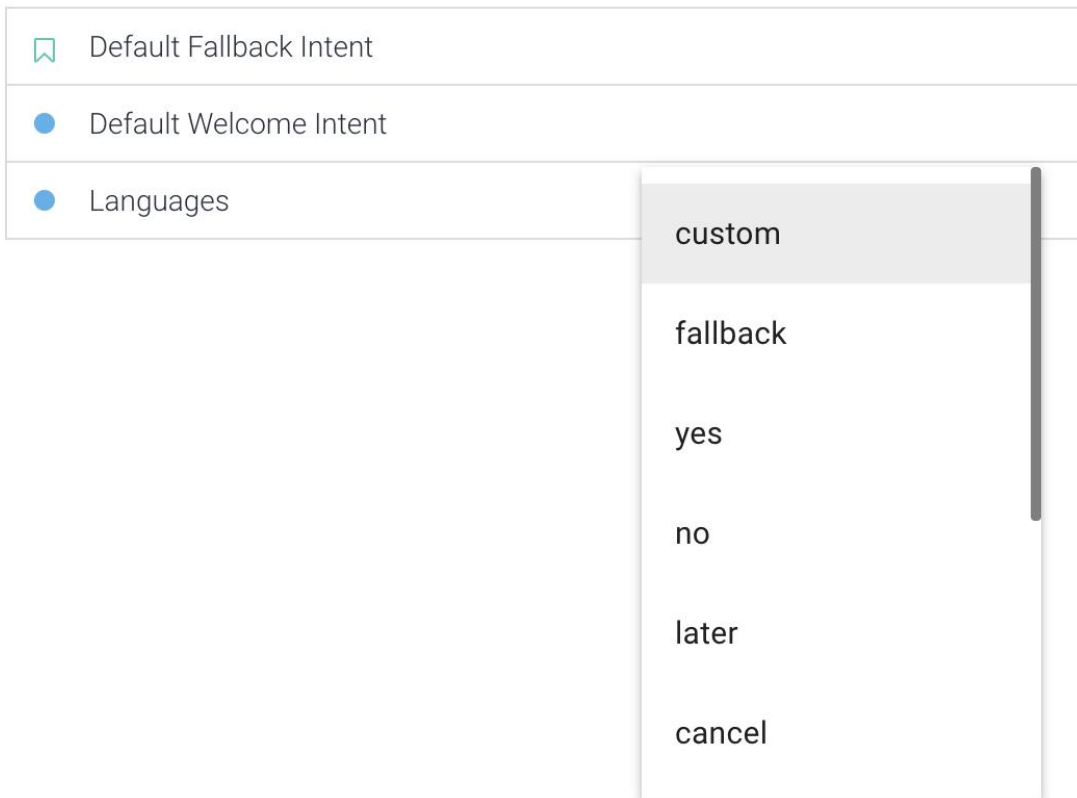
 Default Fallback Intent

 Default Welcome Intent

 Languages

Add follow-up intent  

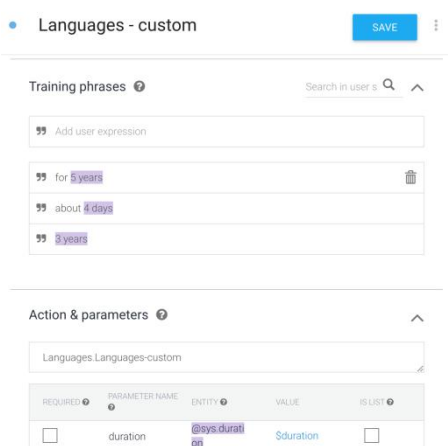
8. Click on **Custom** in the revealed list:



Dialogflow automatically names the follow-up intent "Languages - custom," and the arrow indicates the relationship between the intents.

10.2 Intent matching with follow-up intents

Follow-up intents are only matched after the parent intent has been matched. Since this intent is only matched after the "Languages" intent, we can assume that the user has just been asked the question "How long have you known \$language?". You'll now add Training Phrases indicating users' likely answers to that question.



1. Click on **Intents** in the left menu and then click on the "Languages - custom" intent.
2. Add the following Training Phrases:
 - 3 years
 - about 4 days
 - for 5 years

10.3 Try it out

Try this out in the Dialogflow simulator on the right. First, match the "Languages" intent by entering the query I know French. Then, answer the question How long have your known \$language? with about 2 weeks.

I know French	about 2 weeks								
See how it works in Google Assistant. Agent Domains USER SAYS I know French COPY CURL DEFAULT RESPONSE Wow! I didn't know you knew French. How long have you known French? PLAY CONTEXTS languages-followup RESET CONTEXTS INTENT Languages ACTION Not available <table border="1"> <thead> <tr> <th>PARAMETER</th> <th>VALUE</th> </tr> </thead> <tbody> <tr> <td>language</td> <td>French</td> </tr> </tbody> </table>	PARAMETER	VALUE	language	French	See how it works in Google Assistant. Agent Domains USER SAYS about 2 weeks COPY CURL DEFAULT RESPONSE Not available CONTEXTS languages-followup RESET CONTEXTS INTENT Languages - custom ACTION Languages.Languages-custom <table border="1"> <thead> <tr> <th>PARAMETER</th> <th>VALUE</th> </tr> </thead> <tbody> <tr> <td>duration</td> <td>{"amount":2,"unit":"wk"}</td> </tr> </tbody> </table>	PARAMETER	VALUE	duration	{"amount":2,"unit":"wk"}
PARAMETER	VALUE								
language	French								
PARAMETER	VALUE								
duration	{"amount":2,"unit":"wk"}								

Despite there being no response for the second query ("about 2 weeks"), we can see our query is matched to the correct intent ("Languages - custom") and the duration parameter is correctly parsed ("2 weeks").

10.4 Intents and contexts

Now that your follow-up intent is being matched correctly, you need to add a response. In "Languages - custom" you've only asked for the duration the user has known the language, and not the referenced language itself.

To respond with a parameter gathered from the "Languages" intent, you need to know how follow-up intents work. Follow-up intents use contexts to keep track of if a parent intent has been triggered. If you inspect the "Languages" intent, you'll see "Languages-followup" listed as an Output context, prefaced by the number 2:

- Languages**

SAVE

Contexts ?

Add input context

2 Languages-followup

Add output context

After the "Languages" intent is matched, the context "Languages-followup" is attached to

the conversation for two turns. Therefore, when the user responds to the question, "How long have you known \$language?", the context "Languages-followup" is active. Any intents that have the same Input context are heavily favored when Dialogflow matches intents.

Click on Intents in the left navigation and then click on the "Languages - custom" intent.

You can see that the intent has the same input context ("Languages-followup") as the output context of "Languages". Because of this, "Languages - custom" is much more likely to be matched after the "Languages" intent is matched.

• Languages - custom

SAVE

Contexts ?

Languages-followup × Add input context

Add output context ×

10.5 Contexts and parameters

Contexts store parameter values, which means you can access the values of parameters defined in the "Languages" intent in other intents like "Languages - custom".

Add the following response to the "Languages - custom" intent and click the Save button:

- I can't believe you've known #languages-followup.language for \$duration!

Now you can query your agent again and get the proper response. First enter "I know French", and then respond to the question with "1 month".

You should see that the language parameter value is retrieved from the context.

I know French

1 month

See how it works in Google Assistant.

See how it works in Google Assistant.

AgentDomains

AgentDomains

USER SAYS

COPY CURL

I know French

1 month

DEFAULT RESPONSE

PLAY

Wow! I didn't know you knew French. How long have you known French?

I can't believe you've know French for 1 mo

CONTEXTS

RESET CONTEXTS

languages-followup

languages-followup

INTENT

Languages

Languages - custom

ACTION

Languages.Languages-custom

PARAMETER

VALUE

language

French

duration

{ "amount":1, "unit": "mo" }

Now that you've completed your first agent, you also can use Google “Prebuilt Agents” to rich your project voice ability.

Dialogflow

MyfirstAgent

en

Intents

Entities

Knowledge (beta)

Fulfillment

Integrations

Training

History

Analytics

Prebuilt Agents

Small Talk

Docs

Standard Free Upgrade

Support

Account

Logout

Prebuilt Agents

These are pre-built, customizable agents that specialize in different areas of knowledge. Find an agent you want to use, import it into your account and setup a webhook to get the agent responding. You'll have access to all the parts of the agent, so feel free to modify or reuse intents and entities in your projects. Read more about agents [here](#).

Search...

Agent Name

Conversations to change your agents name

Alarm

Set and manage alarms

App Management

Manage your applications, user controlled settings, and preferences

Banking

Banking operations

Car

Interact with a vehicle's systems

Coffee Shop

Order drinks and snacks from a coffee shop

Currency Converter

Get exchange rates for major currencies

Date

Date calculator and holiday lookup

11. Configuration your command words

In order to satisfy user preferences about the voice command words. We support user to configure their own favorite words to control robot to implement actions.

We already built the command words parameters in robot. You just need to configure the related intents to your agent.

Next I will tell you how to configure your command intents in your agent

11.1 How to configure your agent to let robot open/close a specified app

1. First you need to configure the app entity. Create an entity and name “app”.

Note: The entity name must be “app”.

2. Configuration “app” entity contents: input the provided app name (reference system’s app name) as reference value.

Example: The name “movie” is the system app name, the system only can recognition “movie” as value. If you set a non-system name “film” as a reference value, robots will not response, as it can’t parse to the related program. But you can set the film as movie’s synonym. The reference value(app name) also must be corresponding with your agent language.

In a word: the reference value must be system’s app name, other synonym must be filled in synonym blank.

System’s app name: English, HongKong,Japanese,Korean,Turkish

English	Hong Kong	Japanese	Korean	Turkish
Zhi Yin	知音	音をきく	지음	konser
Settings	設置	設定	설정	Ayarlar
Contacts	通訊錄	連絡先	연락처	İletişim
dance	舞蹈	ダンス	춤	dans
Patrol	巡查	巡回	순찰	denetlemek
Hardware Test	硬件檢測	ハードウェアテスト	H/W 테스트	Donanım Testi
Charge	充電	電荷	충전	Şarj et
Wander	自由行走	ワンダー	자율주행	Gezmek
Duck Run	趕鴨仔	ダックラン	술래잡기	Ördek koşusu
Message Center	通知中心	メッセージセンター	메세지 센터	Mesaj merkezi
APP Market	應用市場	APP Store	앱스토어	Uygulama Marketi
Greeter	迎賓	応接	안내	karşılama
Lucky Draw	幸運抽獎	ラッキードロウ	행운 추첨	Şanslı Beraberlik

Upgrade	升級	更新	업그레이드	güncelleme
Product Introduction	產品介紹	製品紹介アプリ	제품 소개	ürün tanıtımı
Movie	電影	映画	영화	film
Permission	允許	許可	권한 관리	izin
Reminders	日程	リマインダー	스케줄 알림	Hatırlatmalar
Camera	相機	カメラ	카메라	kamera
Trumpet	小喇叭	스피커	트럼펫	Trompet
Music	音樂	音楽	음악	Müzik
Photos	圖片庫	写真	갤러리	Fotoğraflar
FileManager	文件管理	ファイルマネージャ	파일 관리	Dosya Yöneticisi
DesktopReplacementTool	案頭替換工具	デスクトップ入れ替わツール	데스크탑 교체 도구	MasaüstüDeğiştirmeAracı

System's app name: Portuguese,Polish,Italian

Portuguese	Polish	Italian
Salon	Towarzysz	Amico Intimo di
Configurações	Ustawienia	Impostazioni
Contactos	Kontakty	Contatti
Dance	Taniec	danza
Patrulha	Patrol	ispezionare
Teste de hardware	Test Sprzętu	Hardware Test
Cobrança	Ładowanie	Carica
Wander	Wędrówka	Vagare
Corrida de pato	Kaczka Run	Prendimi
Centro de Mensagens	Centrum wiadomości	Centro messaggi
APP Market	APP Market	Mercato APP
Bem vindo	Pozdrawiacz	Benvenuto
Questão de Sorte	Lucky Draw	Sorteggio fortunato
Upgrade	Aktualizacja	Aggiornamento
Descrição do Produto	Wprowadzenie produktów	Introduzione al prodotto
O filme	Film	Film
Gestão da autoridade	Zabezpieczenia	Autorizzazione
agenda	Przypomnienia	Promemoria
A câmera	Aparat fotograficzny	Fotocamera
Pequeno Alto – falante	Trąbka	Recitazione
Música	Muzyka	Musica
Galeria de fotos	Zdjęcia	Fotografie
Gestão de documentos	Menedżer plików	gestore di file
Ferramenta de substituição do ecrã	komputery biurkowe zastapienia narzędzi.	La sostituzione di uno strumento del Desktop

System's app name: French,Spanish,German,Danish

French	Spanish	German	Danish
Connaissance	Consonante	Salon	salon

	Ajustes		Indstillinger
	Contactos		Kontakter
La danse	El baile	Tanzen	dans
Patrouille	Patrulla	Patrouille	Undersøg
Test du matériel	Prueba de hardware	Hardware test	Hardware Påvisning
Charge	Cargar	Kosten	Oplade
Wander	Deambular	Wandern	Vandre
Canard run	Corrida de patos	Duck Run	Duck Run
Centre de messagerie	Centro de mensajes	Message Center	Message Center
APP Market	Tienda de app	APP-Markt	APP marked
écoutez ça	Bienvenida	Willkommen	Hos walmarts
Loterie	Sorteo	Glücklicher punktgewinn	heldig træk
Upgrade	Mejorar	Upgrade	opgradering
Présentation du produit	Introducción del producto	Produkte eingeführt	Produkt Introduction
Film	Película	film	film
Gestion des compétences	Permiso	Zugang die	Tilladelse
Le programme	Reminders	agenda	Påmindelser
Appareil photo	Cámara	Die Kamera	kamera
Trompette	Trompeta	Der Kleine lautsprecher	Trumpet
Musique	Música	Musik	musik
Galerie	Fotos	Galerie	Billeder
Gestion des documents	Administrador de archivos	dateimanager	Filhåndtering
Outils de remplacement de bureau	La sustitución de herramientas de escritorio	Die werkzeuge auf der arbeitsfläche ersetzen	desktop - udskiftning af værktøj

Now we can configure intents to let robot to open a specified app if you configured all these apps to your app entity.

3. Create a intent and name "Sanbot_Openapp" . Configure the training phrases to match user utterance. The training phrases must contain an entity reference value or synonym so that it can parse to corresponding value.

• Sanbot_Openapp

name must be "Sanbot_Openapp"

SAVE

Contexts ?



Events ?



Training phrases ?

Configure all the system app into training phrase and parse the app name to entity reference value.

Search training phrases



” Add user expression

” Open contacts

PARAMETER NAME	ENTITY	RESOLVED VALUE	
app	@app	contacts	x

” Open permission

” Open file manager

” Open app market

” Open upgrade

” Open duck run



” Open camera

” Open music

” Open dance

” Open reminders

Action and parameters



Enter action name

REQUIRED ?	PARAMETER NAME ?	ENTITY ?	VALUE	IS LIST ?
☐	app	@app	Sapp	☐
☐	Enter name	Enter entity	Enter value	☐

+ New parameter

Try it now

Set-up Google Assistant integration.

Agent

USER SAYS COPY CURL

open contacts

DEFAULT RESPONSE

Yes, my lord!

OUTPUT AUDIO

0:00 / 0:00

INTENT

Sanbot_Openapp

ACTION

Not available

PARAMETER	VALUE
app	Contacts

Example: “open contacts” can extract contacts and match to app entity’s reference value (contacts). If you add “address book” as contacts synonym in app entity contacts class. You also can configure “open address book” as training phrases, but you need to extract “address book” to match the entity’s app.

Add address book to the entity of contacts class:

Contacts	Contacts	contacts	address book	Enter synonym

Annotated the address book to the app entity:

Add user expression

Open address book

PARAMETER NAME	ENTITY	RESOLVED VALUE
app	@app	address book

Try it out:

Agent

USER SAYS COPY CURL

Open address book

DEFAULT RESPONSE

Yes, my lord!

OUTPUT AUDIO

0:00 / 0:00

INTENT

Sanbot_Openapp

ACTION

Not available

PARAMETER	VALUE
app	Contacts

The final value is same with the Open contacts. So it have the same logic with open contacts.

Note: The app name in the training phrases should be automatically annotated. If it cannot be annotated just choose the app name and select “@app:app” in prompt box.

Now you can configure other system apps to your Sanbot_Openapp intent by repeat the above steps. The same process to build a intent to implement close a specified app. The close an app intent name is “Sanbot_Closeapp”.

11.2 How to configure other command intents

Now you can configure the other command intent. The different is the other command intent no need to configure any entity. You only need configure the intents but make sure the intent name is corresponding with the **Name of intents table**.

A “light on” as an example:

Sanbot_Lighton

This intent name is specified and can not be changed

SAVE

Contexts

Events

Training phrases

Search training phrases

“” Add user expression

“” Turn on light!

“” Light up!

“” Light on!

Action and parameters

Enter action name

REQUIRED	PARAMETER NAME	ENTITY	VALUE	IS LIST
☐	Enter name	Enter entity	Enter value	☐

The robot will respond the phrases “Turn on light”, “light up”, “Light on” to turn on the LED. You can add more training phrases to rich this action.

If you want the robot to say something like: got it, I will do, Yes, my lord etc., you can fill them to response section of your each intent. This will let robot not only do the action also say out the response sentence.

Responses

DEFAULT +

Text or SSML response

1 Yes, my lord!

2 I will do !

Name of intents table:

Definition	Intent Name
control robot forward	Sanbot_Goforward
control robot turn right	Sanbot_Turnright
control robot turn left	Sanbot_Turnleft
control robot turn about	Sanbot_Turnabout
control robot stop	Sanbot_Stop
control robot come to you	Sanbot_Comehere
control robot leave you	Sanbot_Getout
close robot voice	Sanbot_Silent
control robot start to wander	Sanbot_Starttowander
control robot stop to wander	Sanbot_Stopwandering
control robot open projector	Sanbot_Openprojector
control robot close projector	Sanbot_Closeprojector
control robot turn on light	Sanbot_Lighton
control robot turn off light	Sanbot_Lightoff
control robot play music	Sanbot_Playmusic
control robot pause music	Sanbot_Pausemusic
control robot resume music	Sanbot_Resumemusic
control robot close music	Sanbot_Closemusic
control robot play previous music	Sanbot_Previousmusic
control robot play next music	Sanbot_Nextmusic
control robot play movie	Sanbot_Playmovie
control robot pause movie	Sanbot_Pausemovie
control robot resume movie	Sanbot_Resumemovie
control robot close movie	Sanbot_Closemovie
control robot play previous movie	Sanbot_Previousmovie
control robot play next movie	Sanbot_Nextmovie
control robot start dance	Sanbot_Startdancing
control robot pause dance	Sanbot_Pausedancing
control robot resume dance	Sanbot_Resumedancing
control robot stop dancing	Sanbot_Stopdancing
control robot go to charge	Sanbot_Charger
control robot increase volume	Sanbot_Volumeup
control robot decrease volume	Sanbot_Volumedown
control robot open specified app	Sanbot_Openapp
control robot quit specified app	Sanbot_Closeapp
control robot take photo	Sanbot_Photo

Note: The red marked parts cannot apply to Sanbot Max;

12. How to integrate your agent to robot

Now you finished your dialogflow project, but the robot still doesn't know how to work with your dialogflow project, we need to build a link between them.

Sanbot give users an interface to access your dialogflow to the robot. You can follow below steps to integrate your agent to robot.

Creating the project key of your dialogflow project, details please reference below cutline.

Dialogflow Essentials US ▾

Sanbot-En

en en-AU ...

Intents +

Entities +

Knowledge [beta]

Fulfillment

Integrations

Training

Validation

History

Analytics

General Languages ML Settings Export and Import Environments Speech Share Advanced

DESCRIPTION

Allow your app to engage in small talk about a variety of topics.

DEFAULT TIME ZONE

(GMT-5:00) America/New_York

Date and time requests are resolved using this timezone.

GOOGLE PROJECT

Project ID	small-talk-def1a
Service Account	dialogflow-dusreo@small-talk-def1a.iam.gserviceaccount.com

BETA FEATURES

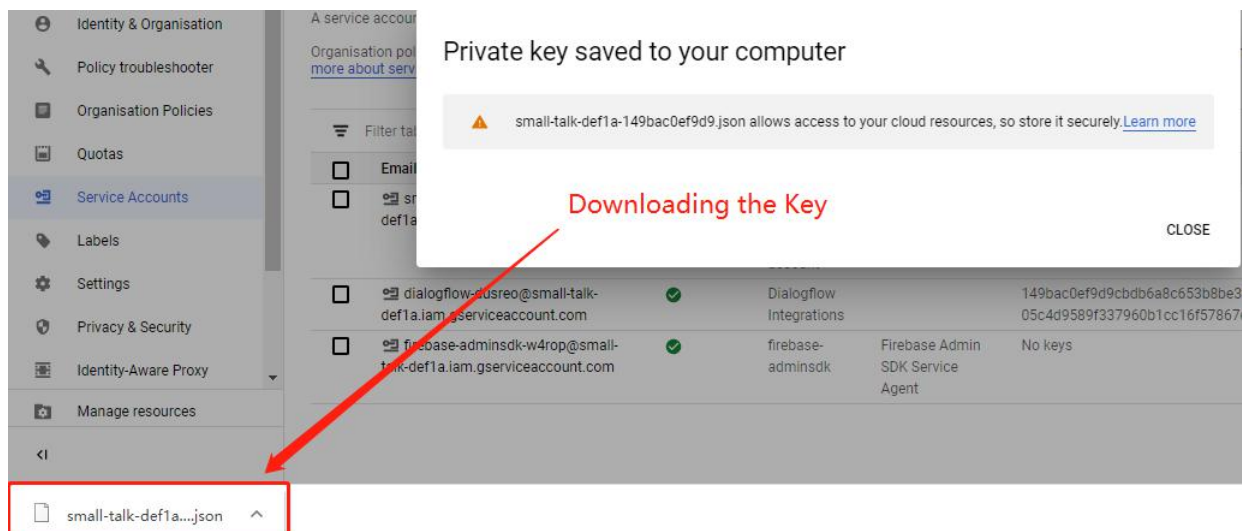
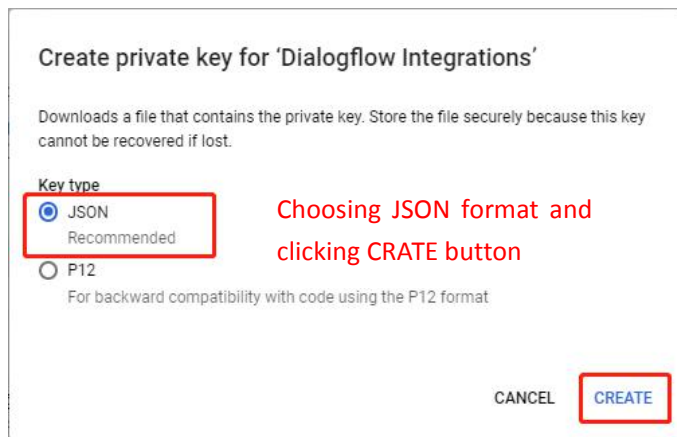
1. Clicking the gear going to the settings page

2. Clicking the link of "Service Account" go to Service accounts window.

Status	Name ↑	Description	Key ID	Key creation date	Actions
✓	App Engine default service account		No keys		⋮
✓	Dialogflow Integrations		6cfaa08484c276b1e8364747301b78ab90851dd8	2 Apr 2019	⋮
✓	firebase-adminsdk	Firebase Admin SDK Service Agent	No keys		⋮

Finding the "Dialogflow Integrations" line on Service accounts page, clicking the 3 dots at actions column, choosing the "Create Key" option to create and saving the key on popped up page.

- Edit
- Disable
- Create key
- Delete



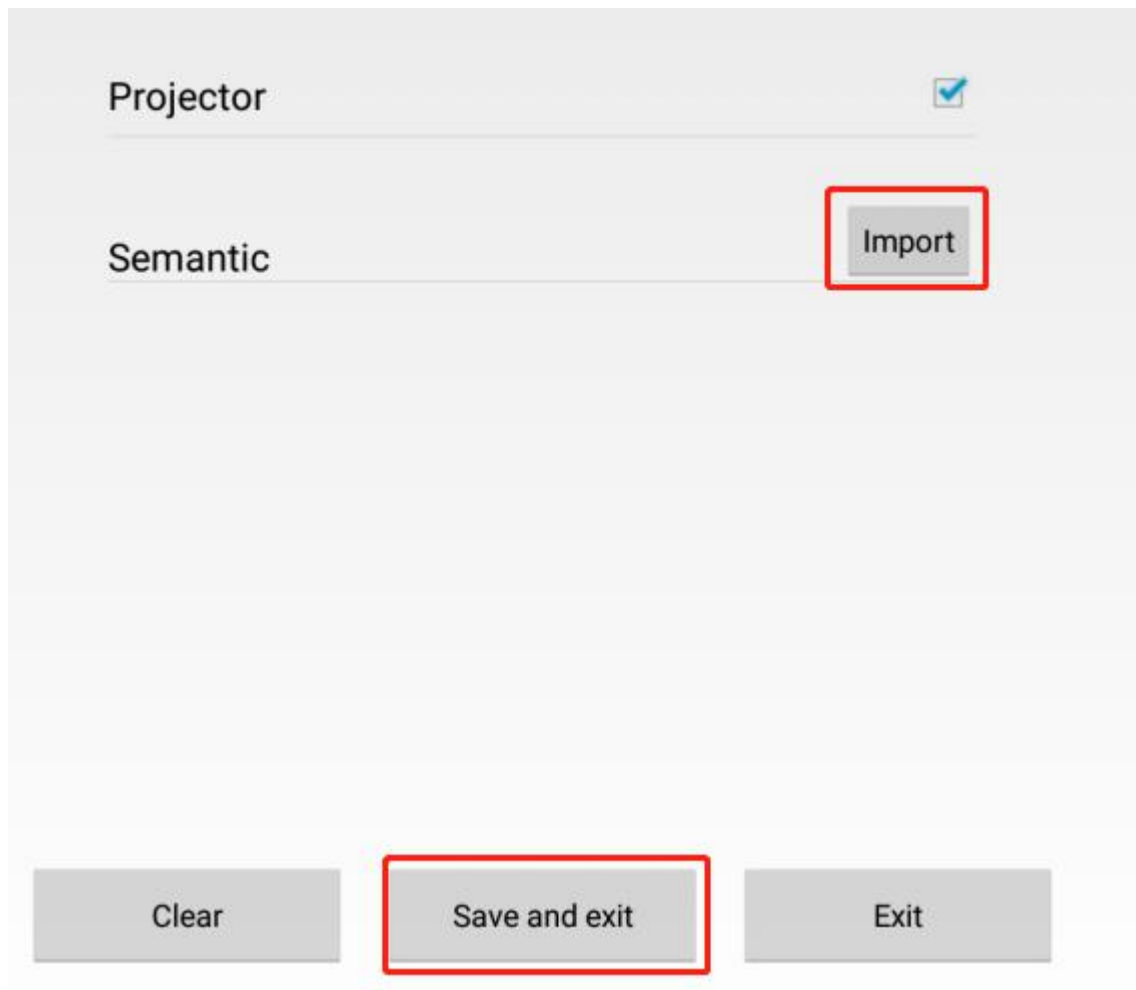
Rename the saved key(Please rename the “xxx.json” to “dialogflow_credentials”).

Note: there is no suffix in the renamed name

Copy the renamed file(dialogflow_credentials) to the root directory of USB flash drive.Insert your USB flash drive to the robot USB port.

Now we forward to robot side:

Power on your robot → run settings app → about → clicking name 8 - 12 times continuously → Import your dialogflow key to the “Semantic” → click “Save and exit” to save.



Set the language and region:

Language & Region → Speech understanding (check DialogFlow) → Speech synthesis (check Google) → Reboot the robot.

Now you can have a conversation with robot.

Hope this can help you, thank you for reading.

